Foreground Mask SourceXtractor++ Documentation

foreground_mask_sourcextractor.py

Field	Value
Script	foreground_mask_sourcextractor.py
Location	C:\Users\gordo\Documents\Github\PythonScripts\Foreground Masking\foreground_mask_sourcextractor.py
Document date	2026-07-05
Documentation scope	Purpose, SourceXtractor++ execution model, masking algorithm, code flow, command-line options, parameter tuning, outputs, validation, and limitations.

High-Level Overview

This script is a Python-controlled workflow for building foreground/background contaminant masks from SourceXtractor++ segmentation output. It is intended for barred spiral galaxy work where compact stars and background sources must be excluded before measuring bar profiles, Fourier terms, ellipse fits, shoulders, ansae, or bar lengths.

The main output is a binary FITS mask, not a cosmetically cleaned galaxy image. A value of 0 means a usable galaxy pixel, and a value of 1 means a masked contaminant pixel. SourceXtractor++ performs the source detection and segmentation; Python prepares detection images, reads the segmentation map, measures segments, filters unwanted detections, dilates the mask, writes outputs, and creates diagnostics.

- The wrapper can process one FITS image or a folder of FITS images selected by a glob pattern.
- Detection can run on the original science image or on a residual image after subtracting a broad smoothed galaxy model.
- The executable path is configurable so SourceXtractor++ can be run from native Linux, Conda, or WSL2.

- The code is modular so mask transformation through a later deprojection/bar-alignment step can be added separately with nearest-neighbour interpolation.

SourceXtractor++ Boundary

SourceXtractor++ is not a Python package dependency of this project. It must be installed separately and made available as a command-line executable. The Python wrapper checks whether the configured command appears to be available before attempting a real run. Use --dry-run to inspect the command without launching SourceXtractor++.

Item	Purpose
--sourcextractor-cmd	Executable command. Defaults to sourcextractor++. Can be set to a full command such as wsl sourcextractor++.
--sourcex-config	Optional tested SourceXtractor++ configuration file. Use this when your installation requires local detection/deblending/measurement syntax.
--write-default-config	Writes a commented placeholder config beside outputs and passes it to SourceXtractor++. It is a starting point, not a complete science config.
--sourcex-arg	Repeatable extra argument token appended to the SourceXtractor++ command. This is the escape hatch for version-specific option names.
--dry-run	Prints the exact SourceXtractor++ command and exits before detection, useful for adapting to a local installation.

Important: SourceXtractor++ command-line option names can vary by packaged version. The wrapper uses explicit default switch names for the intended products (detection image, FITS catalogue, segmentation image, and optional check images), but the --sourcex-config and --sourcex-arg options are deliberately provided so a local build can be supported without editing the Python code.

Masking Algorithm

1. Load the science FITS image with astropy.io.fits, copy the FITS header, convert the data to floating point, and require a two-dimensional image.

2. Prepare a SourceXtractor++ detection image. In original mode this is the science image with NaNs replaced by a safe median background. In residual mode the script first subtracts a broad, NaN-safe smoothed galaxy model.
3. Write the prepared detection image to a temporary FITS file, or to the output folder when --save-intermediate is used.
4. Build the SourceXtractor++ command with the detection image, output catalogue, segmentation image, optional check images, optional config file, and any extra SourceXtractor++ arguments.
5. Run SourceXtractor++ with subprocess, capturing stdout and stderr. A failed executable or non-zero return code becomes a clear Python exception.
6. Read the SourceXtractor++ segmentation check-image as a labelled FITS array.
7. Measure each non-zero segment in Python: area, bounding box, centroid, peak science-image value, elongation, compactness, distance from galaxy centre, and residual statistics when residual detection is active.
8. Filter detections using configurable cuts so real galaxy structures such as the nucleus, bar, ansae, shoulders, spiral arms, or large residual patches are less likely to be masked.
9. Convert retained segmentation labels to a preliminary binary mask where every non-zero retained segment is a contaminant.
10. Dilate the mask with a circular footprint to include PSF wings around detected compact objects.
11. Preserve NaN science regions separately by excluding non-finite science pixels from the final contaminant mask.
12. Save the final mask as uint8 FITS data and create diagnostic PNGs for visual checking.

Residual detection mode

Residual mode computes R = I - G_smooth, where I is the science image and G_smooth is a broad Gaussian-smoothed galaxy model. The smooth model is built by filtering both the finite-pixel data and a finite-pixel weight image, then dividing the two. This avoids NaN regions contaminating the model. Residual detection is useful when the galaxy body is bright enough that direct source detection might confuse bar, disk, arm, or ring structure with foreground/background sources.

Binary mask definition

The SourceXtractor++ segmentation image S is converted into an initial mask by M0(x,y) = 1 when S(x,y) is a retained positive label and M0(x,y) = 0 otherwise. The final mask is M = dilate(M0, r), where r is --dilation-radius in pixels. The final FITS mask stores 0/1 integer values.

Code Flow

Function	Role
load_fits_image	Reads one 2D FITS image and returns floating-point data plus a copied FITS header.
robust_sigma	Computes robust scatter using median absolute deviation, with a standard-deviation fallback.
make_smooth_galaxy_model	Builds the NaN-safe broad galaxy model used for residual detection.
prepare_detection_image	Selects original or residual detection mode and replaces NaNs with a safe median background before SourceXtractor++ runs.
write_temp_fits	Writes the prepared detection image as float32 FITS for SourceXtractor++.
write_sourcextractor_config	Writes a commented placeholder config when --write-default-config is requested.
check_sourcextractor_available	Checks the first executable token, including WSL-style commands, before launching a real run.
build_sourcextractor_command	Assembles the SourceXtractor++ subprocess command, including outputs, config, check images, and extra args.
run_sourcextractor	Runs SourceXtractor++, captures stdout/stderr, and raises clear errors on failure.
read_segmentation_map	Reads the labelled segmentation FITS image created by SourceXtractor++.
measure_segments	Calculates area, bbox, centroid, peak values, elongation, compactness, residual values, and centre distance.

Function	Role
filter_segments	Applies configurable Python-side cuts and returns a filtered segmentation map plus a CSV-ready decision table.
dilate_mask	Expands the retained source mask using a circular footprint.
save_mask_fits	Writes the final uint8 0/1 mask with the original FITS header and provenance keywords.
make_diagnostic_plots	Writes original, segmentation, mask, overlay, residual, and optional SourceXtractor++ check-image PNGs.
build_foreground_mask	Main reusable one-image pipeline function used by the command-line interface.
main	Handles single-file versus folder mode and prints the final run summary.

Parameter Choices

Option	Default	Guidance
input_path	required	A single FITS image or a folder. Folder mode uses --glob and --limit.
--output-dir	sourcex_mask_outputs	Destination for catalogue, segmentation, mask, CSV, and PNG diagnostics.
--glob	*.fits	Folder-mode pattern. Use NGC*.fits for NGC-only S4G tests.
--limit	None	Process only the first N folder matches. Use this for parameter tuning before a large batch.
--hdu-index	0	FITS HDU containing the science image.

Option	Default	Guidance
--sourcextractor-cmd	sourcextractor++	Executable command. Use a WSL or Conda command if needed.
--sourcex-config	None	Path to a tested SourceXtractor++ config for local detection settings and output columns.
--write-default-config	off	Writes a placeholder config for editing; not a substitute for validated SourceXtractor++ settings.
--sourcex-arg	repeatable	Append version-specific SourceXtractor++ arguments without editing Python.
--detect-on	original	Use original for simple fields; use residual to reduce false detections from the galaxy body.
--smooth-sigma-pixels	15	Residual-mode smoothing width. Increase for large smooth galaxies; too small can absorb compact contaminants into the model.
--dilation-radius	3	Grow mask around retained detections. Use 5-10 for bright star wings; keep smaller for conservative galaxy-interior masking.
--min-area	5	Reject tiny detections/noise islands below this segment area.
--max-area	500	Reject large detections that are more likely galaxy structure or broad residuals unless --mask-large-objects is used.
--max-elongation	None	Optional upper elongation cut. Useful for rejecting arms, streaks, and bar-like residuals.
--min-compactness	None	Optional lower compactness cut, area divided by bounding-box area. Helps reject filamentary or diffuse detections.

Option	Default	Guidance
--mask-large-objects	off	If enabled, the --max-area rejection is disabled. Use cautiously, mainly for large foreground stars.
--exclude-central-radius	10	Reject detections inside this pixel radius from the configured galaxy centre. Protects the nucleus and nuclear rings.
--galaxy-centre	auto	auto means image centre; none disables central distance logic; x,y supplies explicit pixel coordinates.
--save-intermediate	off	Keep detection input and optional SourceXtractor++ check-image FITS outputs in the output directory.
--overwrite	off	Required to replace existing catalogue, segmentation, or mask FITS outputs.
--dry-run	off	Print command and stop before SourceXtractor++ detection. Best first step on a new machine.

Starting recipes

- First local test: run with --dry-run to inspect the SourceXtractor++ command before detection.
- Conservative galaxy-structure protection: --detect-on residual --detection settings in the SourceXtractor++ config, plus --max-area 300 --dilation-radius 3.
- Bright foreground-star masking: --detect-on residual --dilation-radius 5 or 8, and consider --mask-large-objects only after inspecting overlays.
- If the nucleus or bar centre is being masked: increase --exclude-central-radius or provide --galaxy-centre x,y.
- If spiral arms or ansae are being masked: reduce --max-area, set --max-elongation, set --min-compactness, or tune SourceXtractor++ detection thresholds upward.

Command-Line Use

Task	Command
Inspect command only	python "Foreground Masking/foreground_mask_sourcextractor.py" "input.fits" --dry-run
One S4G image	python "Foreground Masking/foreground_mask_sourcextractor.py" "D:\Dropbox\Public Documents\UCLAN\MSc Research\Erwin\s4g_images_36um\NGC1097.phot.1.fits" --output-dir "D:\Dropbox\Public Documents\UCLAN\MSc Research\sourcex_mask_examples" --detect-on residual --overwrite
Folder test	python "Foreground Masking/foreground_mask_sourcextractor.py" "D:\Dropbox\Public Documents\UCLAN\MSc Research\Erwin\s4g_images_36um" --glob "NGC*.fits" --limit 5 --output-dir "D:\Dropbox\Public Documents\UCLAN\MSc Research\sourcex_mask_examples" --detect-on residual --overwrite
WSL executable	Append --sourcextractor-cmd "wsl sourcextractor++". If WSL paths are needed by your SourceXtractor++ build, use a WSL-accessible input/output strategy.
Extra SourceXtractor++ switches	Append repeated --sourcex-arg tokens, e.g. --sourcex-arg "--some-option" --sourcex-arg "value".

Outputs

Output suffix	Meaning
***_sourcex_catalog.fits**	SourceXtractor++ output catalogue. Keep this for audit even though the mask is the main science product.
***_sourcex_segmentation.fits**	SourceXtractor++ labelled segmentation check-image. Non-zero labels are candidates before Python filtering.

Output suffix	Meaning
***_foreground_mask.fits**	Final science mask. uint8 FITS image with 0 = usable pixel and 1 = masked contaminant.
***_sourcex_segments.csv**	Python segment-measurement table with area, bbox, centroid, peak, elongation, compactness, distance, and kept flag.
***_original.png**	Percentile/asinh display of the input science image.
***_segmentation_preview.png**	Filtered SourceXtractor++ segmentation labels used for the final mask.
***_foreground_mask_preview.png**	Final binary mask preview.
***_masked_preview.png**	Original image with final contaminant mask overlay.
***_detection_diagnostics.png**	Side-by-side original and mask-overlay view for fast visual checking.
***_residual.png**	Residual detection image when --detect-on residual is used.
***_sourcex_background.png**	Optional background check-image PNG if SourceXtractor++ writes the requested FITS product.
***_sourcex_filtered.png**	Optional filtered check-image PNG if SourceXtractor++ writes the requested FITS product.
***_sourcex_thresholded.png**	Optional thresholded check-image PNG if SourceXtractor++ writes the requested FITS product.

Validation and Caveats

- Begin each new SourceXtractor++ installation with --dry-run and a single galaxy before processing a folder.
- Open *_detection_diagnostics.png first. The mask should identify compact bright contaminants without blanketing the bar, ansae, shoulders, nucleus, or spiral arms.

- Open *_segmentation_preview.png and *_sourcex_segments.csv when tuning area, elongation, compactness, and central-exclusion parameters.
- Check masked fraction relative to finite science pixels. A sudden high fraction usually means SourceXtractor++ or Python filtering is selecting galaxy structure.
- Treat --mask-large-objects as a deliberate exception for large foreground stars, not a default for bar-profile science.
- If a later deprojection/bar-alignment transform is applied to the image, transform the mask using nearest-neighbour interpolation only.

Current limitations

- The wrapper cannot guarantee that default SourceXtractor++ switches match every installed SourceXtractor++ version. Use --dry-run, --sourcex-config, and --sourcex-arg to adapt locally.
- The script does not yet perform the later mask deprojection/bar-alignment transform.
- Folder mode is serial and writes all outputs into one output directory.
- NaN science pixels are excluded from the contaminant mask rather than encoded as contaminants. Carry a separate finite-data mask if downstream code needs to distinguish no-data from usable unmasked pixels.
- This workflow does not create a cleaned science image; it is intentionally mask-first.